

中山大学计算机学院

人工智能

本科生实验报告

(2022 学年春季学期)

课程名称: Artificial Intelligence

教学班级	人工智能 (饶洋辉老师)	专业 (方向)	地理信息科学 (辅修)
学号	23306103	姓名	赵义凯

一、实验题目

利用 PyTorch 框架, 构建一中药图片分类的 CNN 模型。数据包括 5 类图片: baihe、dangshen、gouqi、huaihua、jinyinhua。分别将训练集用于模型训练, 测试集用于性能评估。训练过程中展示 Loss 曲线和准确率曲线。

二、实验内容

1.1 算法原理

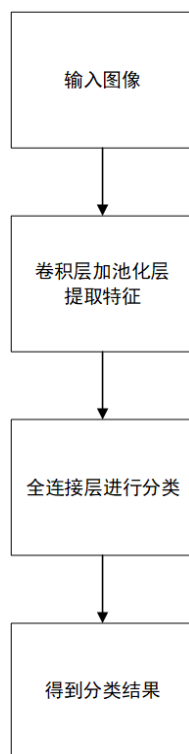
卷积神经网络 (CNN) 是一种深度前馈神经网络, 广泛应用于图像识别与分类任务中。CNN 主要由卷积层 (Convolutional Layer)、激活函数 (如 ReLU)、池化层 (Pooling Layer) 和全连接层 (Fully Connected Layer) 组成。卷积层通过多个局部感受野提取图像的空间特征, 池化层用于降低维度、减少计算量, 而全连接层则将提取到的特征用于最终分类。

在训练过程中, CNN 使用反向传播算法计算损失函数 (如交叉熵损失) 对网络参数 (卷积核权重和全连接层权重) 的梯度, 并通过优化器 (如 Adam) 不断更新参数以最小化损失。通过多轮迭代训练, 模型能够学习到不同类别图像之间的差异特征。

在本实验中, 输入为尺寸为 224×224 的彩色中药图像, 经过 4 层卷积 + 池化层提取特征后, 通过全连接层输出图像所属类别 (如白菊、枸杞、金银花等) 的预测结果, 实现对中药图像的自动分类。

数据集划分: 提供的数据集中, test 中每个类别只有 1-2 张图像, 不利于对模型进行评价。于是我手动从 train 中选取部分图像, 保持在 train 和 test 比例为 8: 2, 方便评价模型。

1.2 流程图



2.关键代码展示（带注释）

在实验中，我主要进行了三方面的代码尝试：首先，定义了一个包含三个卷积层的 **CNN** 模型，并采用给定的图像预处理函数进行基础模型训练；接着，将该 **CNN** 模型与上次实验构建的 **MLP** 模型进行性能对比分析；最后，通过 **transforms** 模块实现了数据增强（如随机翻转、旋转等），并针对增强数据的特性，对网络架构进行了优化——增加一层卷积层以提取更抽象的特征表示，同时配置学习率调度器并延长训练轮次，从而保证模型在数据多样性提升的情况下仍能稳定收敛。



1) 三层的网络

```
# 模型定义
class TCM_CNN(nn.Module):
    def __init__(self):
        super(TCM_CNN, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2), # 112x112
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2), # 56x56
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2) # 28x28
        )
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 28 * 28, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, len(classes_name))
        )
    def forward(self, x):
        x = self.conv(x)
        x = self.fc(x)
        return x

cnn = TCM_CNN()
optimizer = torch.optim.Adam(cnn.parameters(), lr=LR)
loss_func = nn.CrossEntropyLoss()
# 记录损失和准确率
train_loss_list = []
train_acc_list = []
test_acc_list = []
# -----训练过程-----
for epoch in range(EPOCH):
    cnn.train()
    total_loss = 0
    total_correct = 0
    total_samples = 0
    for step, (b_x, b_y) in enumerate(train_loader):
        output = cnn(b_x)
        loss = loss_func(output, torch.tensor(b_y))
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
        pred_y = torch.max(output, 1)[1]
        total_correct += (pred_y == torch.tensor(b_y)).sum().item()
        total_samples += len(b_y)
        if step % 5 == 0:
            print(f"Epoch [{epoch+1}], Step [{step}], Loss: {loss.item():.4f}")
    acc = total_correct / total_samples
    train_loss_list.append(total_loss)
    train_acc_list.append(acc)
    # -----测试精度计算-----
    cnn.eval()
    test_correct = 0
    test_total = 0
    with torch.no_grad():
        for test_images, test_labels in test_loader:
            outputs = cnn(test_images)
            predicted = torch.max(outputs, 1)[1]
            test_total += len(test_labels)
            test_correct += (predicted == torch.tensor(test_labels)).sum().item()
    test_acc = test_correct / test_total
    test_acc_list.append(test_acc)
    print(f"Epoch [{epoch+1}] Finished | Train Loss: {total_loss:.4f} | Train Acc: {acc:.4f} | Test Acc: {test_acc:.4f}")
```

2) 四层网络定义、transform 图像处理及学习率调度器

由于除了这几部分外其余部分的代码几乎一致，故我只列出这几部分代码。



```
# -----数据增强与预处理-----
transform_train = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])
transform_test = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])
```

图 1 transforms 图像预处理

```
class TCM_CNN(nn.Module):
    def __init__(self):
        super(TCM_CNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(16),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # 112x112
            nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # 56x56
            nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # 28x28
            nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # 14x14
        )
        self.classifier = nn.Sequential(
            nn.Dropout(p=0.5),
            nn.Linear(128 * 14 * 14, 1024),
            nn.ReLU(inplace=True),
            nn.Dropout(p=0.5),
            nn.Linear(1024, 512),
            nn.ReLU(inplace=True),
            nn.Linear(512, num_classes),
        )
        self._initialize_weights()
    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x
    def _initialize_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
                if m.bias is not None:
                    nn.init.constant_(m.bias, 0)
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.constant_(m.weight, 1)
                nn.init.constant_(m.bias, 0)
            elif isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, 0, 0.01)
                nn.init.constant_(m.bias, 0)
```

图 2 四层网络定义

```
scheduler = StepLR(optimizer, step_size=6, gamma=0.5)
```

图 3 学习率调度器设置

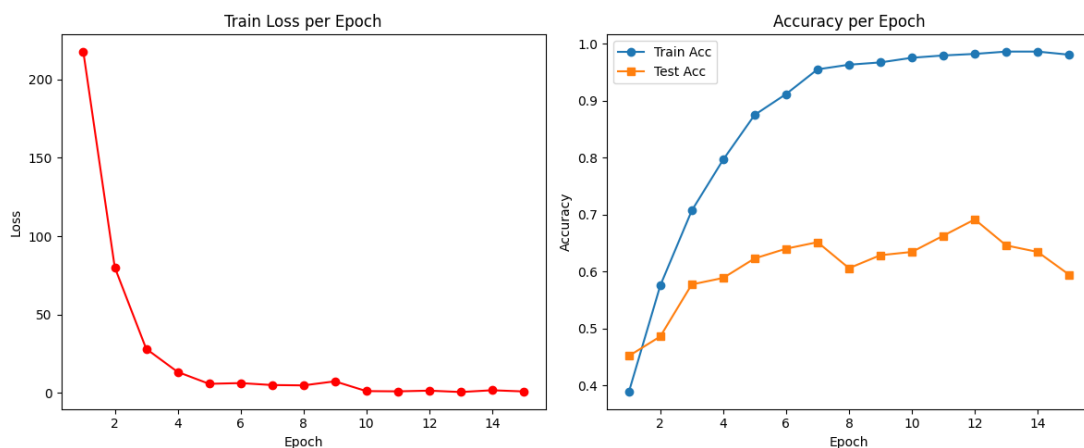
1. 创新点&优化（如果有）

- 使用 matplotlib 库对训练过程中的损失、精度等进行可视化处理
- 使用 transforms 对图像进行旋转、折叠等增强处理，并且加深 cnn 网络层数，保证网络学习到深层次的特征
- 使用学习率调度器，防止训练过拟合

三、 实验结果及分析

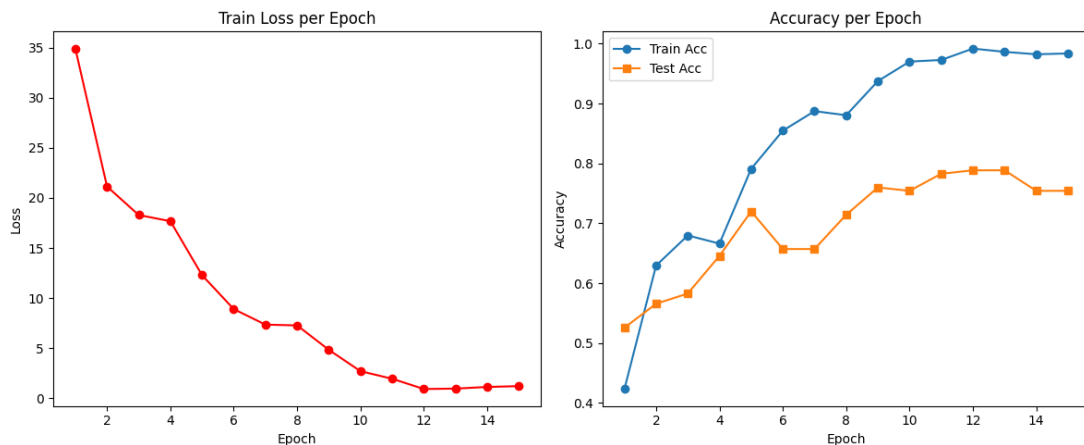
1. 实验结果展示示例（可图可表可文字，尽量可视化）

- mlp 模型训练过程曲线图



分析：训练损失快速下降，但测试准确率波动大，过拟合显著，表明全连接网络难以捕捉图像空间特征。

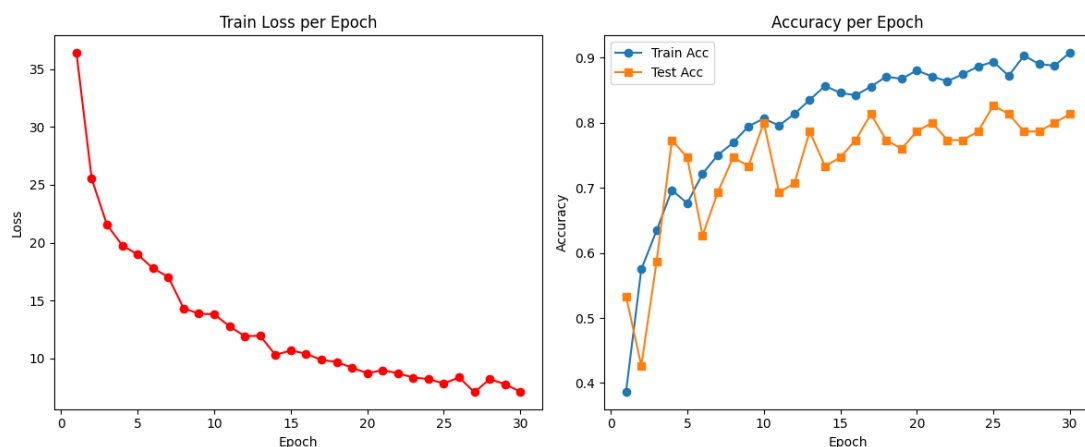
- 3 层 cnn 模型训练过程曲线图





分析：测试准确率提升至 77.53%，但训练准确率接近 100%，仍存在较重过拟合，说明浅层网络表达能力有限。

- c) 4 层 cnn 模型（图像增强、轮次增多、学习率调度）训练过程曲线图



分析：测试准确率达 81.33%，训练与测试准确率差距缩小，数据增强和学习率调度有效缓解过拟合，深层网络提取到更鲁棒的特征。

2. 评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

- a) Mlp 模型指标

```
Epoch [7/15] Finished | Train Loss: 4.9749 | Train Acc: 0.9552 | Test Acc: 0.6514
Epoch [8/15] Finished | Train Loss: 4.7460 | Train Acc: 0.9634 | Test Acc: 0.6057
Epoch [9/15] Finished | Train Loss: 7.3323 | Train Acc: 0.9674 | Test Acc: 0.6286
Epoch [10/15] Finished | Train Loss: 1.0943 | Train Acc: 0.9756 | Test Acc: 0.6343
Epoch [11/15] Finished | Train Loss: 0.9371 | Train Acc: 0.9796 | Test Acc: 0.6629
Epoch [12/15] Finished | Train Loss: 1.3902 | Train Acc: 0.9824 | Test Acc: 0.6914
Epoch [13/15] Finished | Train Loss: 0.5148 | Train Acc: 0.9864 | Test Acc: 0.6457
Epoch [14/15] Finished | Train Loss: 1.6405 | Train Acc: 0.9864 | Test Acc: 0.6343
Epoch [15/15] Finished | Train Loss: 0.8803 | Train Acc: 0.9810 | Test Acc: 0.5943
```

- b) 3 层 cnn 模型指标

```
Epoch [14], Step [15], Loss: 0.0048
Epoch [14], Step [20], Loss: 0.0091
Epoch [14] Finished | Train Loss: 1.1387 | Train Acc: 0.9824 | Test Acc: 0.7543
Epoch [15], Step [0], Loss: 0.0224
Epoch [15], Step [5], Loss: 0.0193
Epoch [15], Step [10], Loss: 0.0353
Epoch [15], Step [15], Loss: 0.0466
Epoch [15], Step [20], Loss: 0.0304
Epoch [15] Finished | Train Loss: 1.2236 | Train Acc: 0.9837 | Test Acc: 0.7543
```

- c) 4 层 cnn 模型（图像增强、轮次增多、学习率调度）指标

```
Epoch [29], Step [25], Loss: 0.2241
Epoch [29] | Train Loss: 7.7525 | Train Acc: 0.8877 | Test Acc: 0.8000 | LR: 0.000063
Epoch [30], Step [0], Loss: 0.2074
Epoch [30], Step [5], Loss: 0.3068
Epoch [30], Step [10], Loss: 0.1693
Epoch [30], Step [15], Loss: 0.1730
Epoch [30], Step [20], Loss: 0.2328
Epoch [30], Step [25], Loss: 0.1240
Epoch [30] | Train Loss: 7.0967 | Train Acc: 0.9080 | Test Acc: 0.8133 | LR: 0.000031
```



模型	训练准确率	测试准确率	过拟合程度
MLP 模型	98.24%	59.43%	高
3 层 CNN 模型	98.37%	75.43%	中高
4 层 CNN 模型	90.80%	81.33%	中低

对这些指标依次分析，可以看出 MLP 模型和 3 层网络模型在训练集上的精度都很高（接近 1），但是其在训练集上的表现都很差，可以看出模型出现了过拟合的情况，但是在此上 cnn 模型却在测试集上的表现比 MLP 模型相对更好一些，这也看出了 CNN 模型比 MLP 模型更适用于图像分类任务。接着看 4 层 CNN 模型，可以看出其过拟合现象有所减轻，在测试集上的表现还不错，符合我们的要求。

四、 思考题

1. 为什么 CNN 在图像分类中优于 MLP?

CNN 的卷积层通过权值共享和局部连接减少参数数量，避免过拟合；池化层引入平移不变性，更适合提取图像空间特征。而 MLP 全连接结构对图像数据的空间关系建模效率低，易过拟合。

2. 数据增强对模型性能的影响机制是什么?

数据增强通过生成虚拟样本（如翻转、旋转）扩大训练集多样性，迫使模型学习更通用的特征（如形状、纹理而非背景颜色），从而提升泛化能力，降低对特定样本的依赖。

3. 如何进一步优化中药分类模型?

可以引用预训练的更深层的网络（如 Resnet 等），同时增加数据集的量。可以获得更好的中药分类模型。

五、 参考资料

- [1]. [Pytorch 实现中药材\(中草药\)分类识别\(含训练代码和数据集\)_中草药数据集-CSDN 博客](#)
- [2]. Lab8 参考文档